



UMCS

UNIwersytet Marii Curie-Skłodowskiej
w Lublinie

Wydział Matematyki, Fizyki i Informatyki

Kierunek: **informatyka**

Daniel Wiszniowski

nr albumu: 318635

Projekt i realizacja bezakumulatorowych czujników klimatycznych zasilanych słonecznie

Design and development of battery-free
solar-powered climate sensors

Praca licencjacka

napisana w Katedrze Oprogramowania Systemów Informatycznych

Instytutu Informatyki i Matematyki UMCS

pod kierunkiem **dr hab. Beaty Byliny, prof. UMCS**

Lublin 2026

Spis treści

Wstęp	5
1 Część sprzętowa	7
1.1 Założenia projektowe	7
1.1.1 Niski pobór mocy	7
1.1.2 Zastosowanie paneli fotowoltaicznych	7
1.1.3 Superkondensatory a akumulatory	7
1.1.4 Komunikacja radiowa	7
1.2 Podstawowe elementy układu	8
1.2.1 Sekcja ładowania superkondensatorów	8
1.2.2 Układ regulacji napięcia	9
1.2.3 Mikrokontroler ATmega328P	10
1.2.4 Magistrała I^2C	11
1.2.5 Układ komunikacji radiowej	12
1.3 Elementy pomiarowe	12
1.3.1 Barometr, higrometr i termometr	12
1.3.2 Moduł z licznikiem Geigera-Müllera	13
1.3.3 Moduł pomiaru natężenia światła	13
1.4 Elementy dodatkowe	13
1.4.1 Magistrała 1-Wire	13
1.4.2 Wolne wyprowadzenia GPIO	13
2 Oprogramowanie	15
2.1 Język Rust w programowaniu mikrokontrolerów	15
2.2 Narzędzia i biblioteki	15
2.3 Obsługa elementów pomiarowych	15
2.3.1 Moduł AHT20 — czujnik temperatury i wilgotności względnej	15
2.3.2 Moduł BMP280	19
2.3.3 Moduł VEML7700	19
2.3.4 Moduł z licznikiem Geigera-Müllera	19

2.4	Komunikacja radiowa	20
2.4.1	Serializacja danych	20
2.4.2	Zabezpieczenie danych	20
2.4.3	Transmisja danych	20
3	Testy	21
3.1	Zasięg komunikacji radiowej	21
3.2	Pobór prądu	21
3.3	Czas pracy	21
	Podsumowanie	23
	Spis listingów	25
	Spis tabel	27
	Spis rysunków	29
	Bibliografia	31

Wstęp

Tu treść wstępu (który piszemy na końcu, po napisaniu całej pracy).

Rozdział 1

Część sprzętowa

1.1 Założenia projektowe

1.1.1 Niski pobór mocy

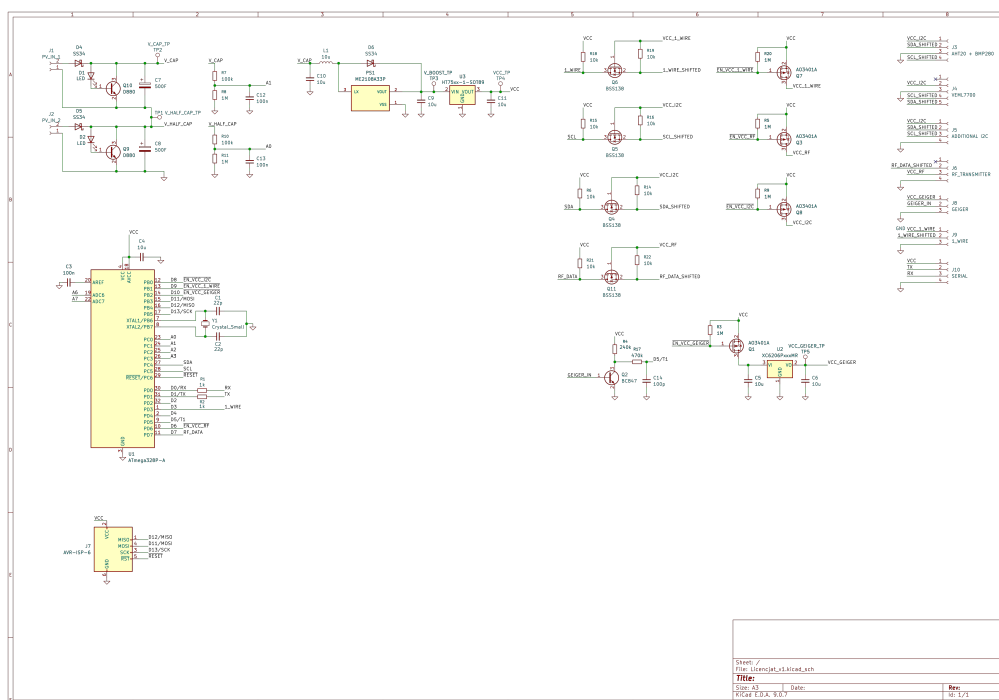
1.1.2 Zastosowanie paneli fotowoltaicznych

1.1.3 Superkondensatory a akumulatory

1.1.4 Komunikacja radiowa

1.2 Podstawowe elementy układu

Schemat elektryczny, widoczny na rysunku 1.1, został podzielony na segmenty pełniące konkretne funkcje. [TODO: lepszy rysunek]

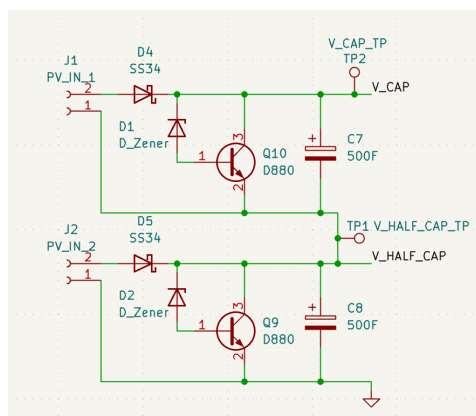


Rysunek 1.1: Schemat elektryczny części sprzętowej projektu.

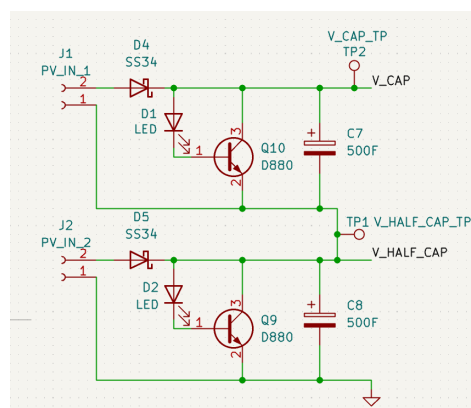
1.2.1 Sekcja ładowania superkondensatorów

Zastosowanie paneli fotowoltaicznych jako jedyne źródła energii do ładowania superkondensatorów wiąże się z określonymi ograniczeniami. Ze względu na zmienne warunki nasłonecznienia, wynikające z czynników zewnętrznych, takich jak okresowe zacinienie panelu przez elementy otoczenia lub zmiany pory dnia, napięcie generowane na wyjściu panelu fotowoltaicznego nie ma charakteru stałego. W celu zapobieżenia wystąpieniu niepożądanych i nieodwracalnych zmian w strukturze superkondensatora konieczne jest ograniczenie napięcia ładowania do wartości nieprzekraczającej napięcia znamionowego [8], które w analizowanym układzie wynosi 2,7 V. W związku z powyższym napięcie pochodzące z ogniwa fotowoltaicznego jest regulowane za pomocą regulatora bocznikowego (ang. *shunt regulator*), zrealizowanego w postaci tranzystora zwierającego ze sobą wyjścia ogniwa oraz diody sterującej jego pracą. Dodatkowo, w celu zabezpieczenia panelu fotowoltaicznego przed przepływem prądu wstecznego, na jego dodatnim wyjściu zastosowano diodę Schottky'ego.

W początkowej wersji regulatora bocznikowego jako element sterujący pracą tranzystora zastosowano diodę Zenera. Jego schemat przedstawiono na rysunku 1.2. W wyniku przeprowadzonych testów rozwiązanie to zostało jednak zastąpione diodą LED (rysunek 1.3), co pozwoliło na zwiększenie stabilności pracy regulatora w zakresie prądu przewodzonego przez tranzystor. Porównanie parametrów obu rozwiązań przedstawiono w tabeli 1.1.



Rysunek 1.2: Sekcja ładowania superkondensatorów z użyciem diod zenera (D1, D2)



Rysunek 1.3: Sekcja ładowania superkondensatorów z użyciem diod LED (D1, D2)

I_E [mA]	U_{Wy} Zener [V]	U_{Wy} LED [V]
5	1,80	2,38
20	2,09	2,45
50	2,25	2,50
100	2,40	2,53
200	2,65	2,57
500	3,00	2,68

Tabela 1.1: Napięcie wyjściowe U_{Wy} regulatora bocznikowego przy użyciu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego.

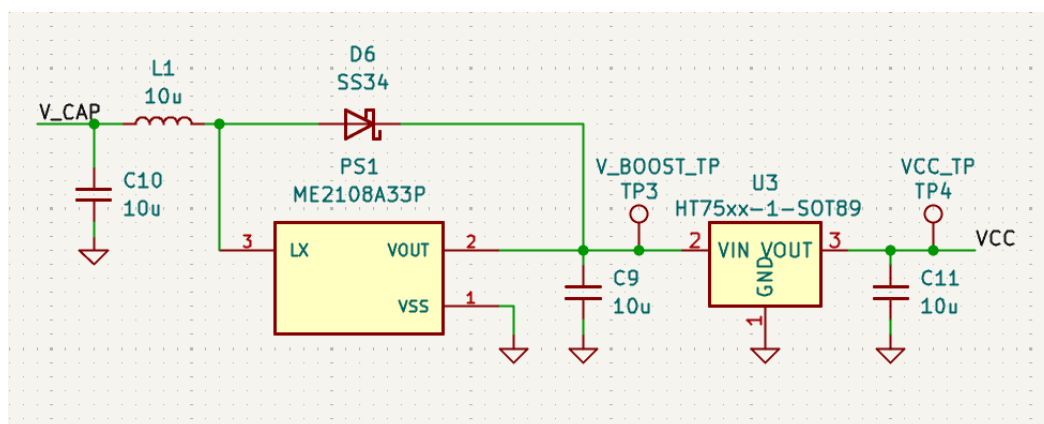
1.2.2 Układ regulacji napięcia

Układ regulacji napięcia, przedstawiony na rysunku 1.4, składa się z 2 połączonych ze sobą części.

Pierwszą z nich jest przetwornica typu step-up, zbudowana w oparciu o układ ME210833P. Jej zastosowanie pozwala na zapewnienie zasilania gdy napięcie na połą-

czonych szeregowo superkondensatorach spadnie poniżej 3,3V. Pozwoli to na rozładowanie ich do łącznego napięcia około 1V, co wynika z ograniczeń układu sterującego przetwornicy [7]. W momencie gdy napięcie wejściowe przetwornicy przekroczy 3,3V, zostanie ona pominięta, przez diodę Schottky’ego podłączoną pomiędzy jej wejściem i wyjściem.

Napięcie jest następnie ograniczane do wartości 3.3V z użyciem regulatora liniowego HT7333 o niskim spadku napięcia (rzędu 80mV [6]), i niskim prądzie spoczynkowym (o maksymalnej wartości 8 μ A [6]).



Rysunek 1.4: Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy.

1.2.3 Mikrokontroler ATmega328P

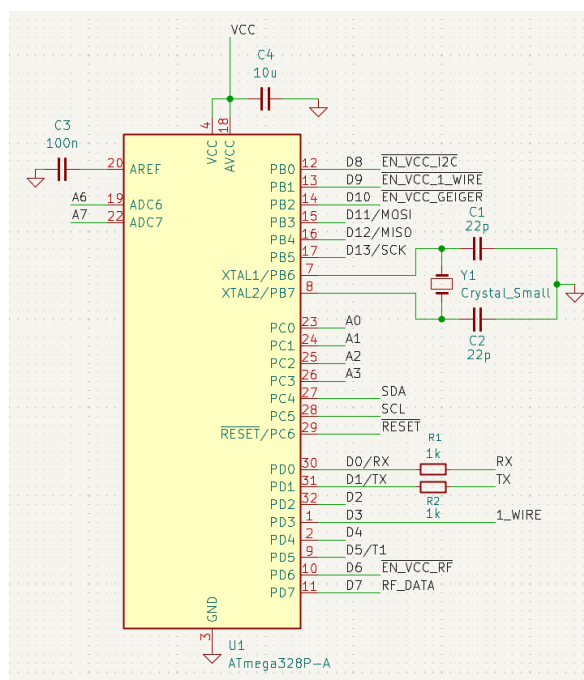
Głównym elementem układu jest mikrokontroler ATmega328P, oparty o 8-bitowy procesor z architekturą AVR firmy Atmel, posiadający 32KB wbudowanej pamięci typu flash [5]. Najważniejszymi jego cechami, w kontekście tego projektu, są niski pobór mocy, możliwość uśpienia oraz wbudowane konwertery analogowo-cyfrowe [5]. Ze względu na popularność mikrokontrolerów rodziny AVR i wykorzystanie ich w platformach takich jak Arduino UNO [2] czy Arduino NANO [3], posiadają one rozbudowany ekosystem zapewniający narzędzia potrzebne do ich oprogramowania.

Mikrokontroler został wyposażony w 3,6864 MHz oscylator kwarcowy. Wartość ta została wybrana ze względu na wygodę generowania standardowych prędkości UART dla których dzielniki, przy takiej częstotliwości, są liczbami całkowitymi [1].

Schemat połączeń mikrokontrolera widoczny jest na rysunku 1.5, a opis funkcji portów jest przedstawiony w tabeli 1.2.

Port	Przeznaczenie pinu	Funkcja
PB0	I/O	Sterowanie zasilaniem urządzeń magistrali I^2C .
PC4	I/O	Linia danych magistrali I^2C .
PC5	I/O	Linia sygnału zegarowego magistrali I^2C .
PB1	I/O	Sterowanie zasilaniem urządzeń magistrali 1Wire.
PD3	I/O	Linia danych magistrali 1Wire.
PD6	I/O	Sterowanie zasilaniem dla nadajnika radiowego
PD7	I/O	Linia danych nadajnika radiowego
PB2	I/O	Sterowanie zasilaniem dla modułu radiometrycznego.
PD3	T1	Zliczanie impulsów modułu radiometrycznego.
PC0	ADC	Badanie napięcia na 1 z 2 superkondensatorów.
PC1	ADC	Badanie napięcia połączonych szeregowo superkondensatorów.

Tabela 1.2: Opis funkcji wyprowadzeń mikrokontrolera. Typy portów: I/O - cyfrowy port wejścia/wyjścia, Tn - podłączenie do wewnętrznego licznika n mikrokontrolera, ADC - wejście konwertera analogowo-cyfrowego.



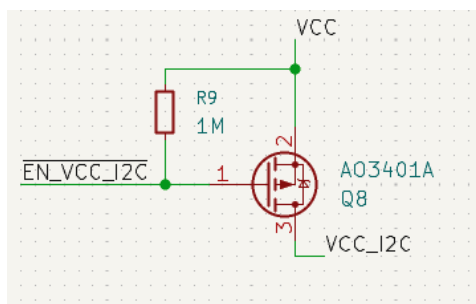
Rysunek 1.5: Mikrokontroler ATmega328P widoczny na schemacie projektu.

1.2.4 Magistrala I^2C

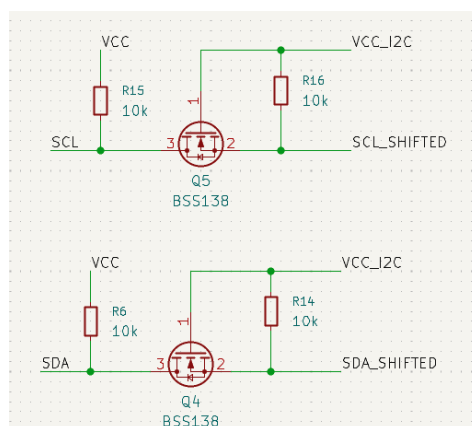
I^2C jest magistralą i protokołem synchronicznym umożliwiającym komunikację pomiędzy urządzeniami z użyciem 2 połączeń: zegarowego (oznaczanego jako SCL) oraz

danych (oznaczanego jako SDA). Wykorzystane w tym celu wyprowadzenia mikrokontrolera widoczne są w tabeli 1.2. W projekcie użyte zostały 3 urządzenia używające tej magistrali: AHT20, BMP280 i VEML7700, które zostaną omówione w rozdziale 1.3.

Dodatkowo, w celu konserwacji energii, linia zasilania urządzeń tej magistrali może zostać odłączona poprzez podanie stanu wysokiego na tranzystor widoczny na rysunku 1.6. Aby zabezpieczyć niezasilone urządzenia magistral przed napięciem, mogącym się pojawić na liniach SDA i SCL, zostały zastosowane konwertery poziomów logicznych (ang. *level shifters*), pokazane na rysunku 1.7. Jednocześnie rezystory w nich zastosowane pełnią funkcję rezystorów podciągających dla linii SDA i SCL, wymaganych przez specyfikację standardu I^2C [9].



Rysunek 1.6: Tranzystor załączający linię zasilania urządzeń magistrali I^2C .



Rysunek 1.7: Konwertery poziomów logicznych dla linii SDA i SCL magistrali I^2C .

1.2.5 Układ komunikacji radiowej

Komunikacja radiowa odbywa się poprzez bezpośrednie sterowanie linią danych modułu nadajnika radiowego. Odpowiedzialność za przygotowanie i buforowanie danych w całości spoczywa na mikrokontrolerze. Z racji dużego poboru mocy przez układ radiowy [TODO: Wstaw pobór mocy], linia zasilania, podobnie jak w przypadku magistrali I^2C , została wyposażona w odcinający przepływ prądu tranzystor i konwerter stanów logicznych dla linii danych, jak pokazano na rysunku [TODO: Wstaw rysunek].

1.3 Elementy pomiarowe

1.3.1 Barometr, higrometr i termometr

Urządzenie zostało wyposażone w 2 moduły znajdujące się na wspólnej płytce:

- BMP280 — mogący mierzyć temperaturę i ciśnienie atmosferyczne;
- AHT20 — mogący mierzyć temperaturę i wilgotność względną.

Obydwa moduły używają magistrali I^2C do komunikacji. [TODO: Rozwiń]

1.3.2 Moduł z licznikiem Geigera-Müllera

Licznik Geigera-Müllera użyty na potrzeby tego projektu, widoczny na rysunku [TODO: Wstaw zdjęcie], jest układem autorstwa inż. Karola Karpowicza i składa się z lampy Geigera-Müllera, wzmacniacza impulsów i generatora wysokiego napięcia. Na jego wyjściu, w momencie detekcji promieniowania, pojawia się impuls o wysokości napięcia zasilania, zadawany na wejście licznika mikrokontrolera. Opis sposobu zliczania impulsów opisany jest w rozdziale [2.3.4](#).

1.3.3 Moduł pomiaru natężenia światła

Za pomiar natężenia światła odpowiedzialny jest moduł VEML7700. Podobnie jak BMP280 i AHT20 komunikuje się z mikrokontrolerem używając protokołu I^2C . Natężenie światła mierzony jest w luksach. [TODO: Rozwiń]

1.4 Elementy dodatkowe

1.4.1 Magistrala 1-Wire

[Nie zaimplementowane]

1.4.2 Wolne wyprowadzenia GPIO

[Nie zaimplementowane]

Rozdział 2

Oprogramowanie

2.1 Język Rust w programowaniu mikrokontrolerów

2.2 Narzędzia i biblioteki

2.3 Obsługa elementów pomiarowych

W celu ograniczenia wykorzystania zasobów pamięciowych mikrokontrolera znaczna część konfiguracji urządzeń pomiarowych realizowana jest na etapie kompilacji. Rozbudowany system typów (ang. *type system*) języka Rust umożliwia zwięzłe i czytelne wyrażenie konfiguracji z wykorzystaniem typów generycznych (ang. *generic types*), cech (ang. *traits*) oraz stałych czasu kompilacji (ang. *const generics*).

2.3.1 Moduł AHT20 — czujnik temperatury i wilgotności względnej

Obsługa modułu AHT20 została zrealizowana w postaci struktury `Aht20` wraz z jej implementacją. Komunikacja realizowana jest za pośrednictwem magistrali I^2C poprzez wysyłanie dedykowanych komend sterujących. Moduł nie wymaga przesyłania danych konfiguracyjnych w trakcie działania. Zgodnie z notą katalogową producenta [4] urządzenie obsługuje następujący zestaw komend:

- **Inicjalizacja** — wymuszenie procedury kalibracji wewnętrznej układu;
- **Wyzwolenie pomiaru** — zainicjowanie akwizycji danych oraz ich zapis do wewnętrznej pamięci urządzenia;
- **Odczyt statusu** — weryfikacja stanu kalibracji układu oraz gotowości do przeprowadzenia kolejnego pomiaru.

Procedura inicjalizacji modułu, przedstawiona na listingu 2.1, rozpoczyna się od odczekania czasu rozruchu wynoszącego 40ms, po którym następuje odczyt rejestru statusu. W przypadku braku flagi kalibracji wysyłana jest komenda inicjalizacji, a następnie wprowadzane jest opóźnienie 10ms niezbędne do jej wykonania:

Listing 2.1: Inicjalizacja modułu AHT20

```

1 pub fn init<D: DelayNs>(
2     &self, i2c: &mut I2c,
3     delay: &mut D
4 ) -> Result<(), Aht20Error> {
5     delay.delay_ms(40);
6
7     let status = self.read_status(i2c)?;
8
9     if status & STATUS_CALIBRATED_BIT == 0 {
10         i2c.write(self.address, &COMMAND_INITIALIZE)?;
11         delay.delay_ms(10);
12     }
13
14     Ok(())
15 }
```

Odczyt surowych danych pomiarowych, przedstawiony na listingu 2.2, realizowany jest w trzech etapach. W pierwszym kroku wysyłana jest komenda wyzwolenia pomiaru, po której, zgodnie z wymaganiami noty katalogowej, następuje odczekanie 80ms do czasu jego zakończenia. Następnie odczytywany jest rejestr statusu w celu weryfikacji gotowości układu — jeżeli bit zajętości jest ustawiony, funkcja zwraca błąd `Aht20Error::SensorBusy`. W przypadku pomyślnej weryfikacji dane pomiarowe zostają zaczytane do 7-bajtowego bufora i zwrócone w postaci struktury `Aht20RawData`. [TODO: może skrócić? Bo wsm to opis kodu]

Listing 2.2: Odczyt surowych danych z modułu AHT20

```

1 pub fn read_raw<I: I2c, D: DelayNs>(
2     &self,
3     i2c: &mut I,
4     delay: &mut D,
5 ) -> Result<Aht20RawData, Aht20Error> {
6     i2c.write(self.address, &COMMAND_START_MEASUREMENT)?;
7
8     // Datasheet 5.4 step 3: wait 80 ms, then check busy bit
9     delay.delay_ms(80);
10
11     let status = self.read_status(i2c)?;
12     if status & STATUS_BUSY_BIT != 0 {
```

```

13         return Err(Aht20Error::SensorBusy);
14     }
15
16     let mut bytes = [0u8; 7];
17     i2c.read(self.address, &mut bytes)?;
18
19     Ok(Aht20RawData { bytes })
20 }

```

Konwersja surowych danych odczytanych z modułu na stopnie Celsjusza oraz wilgotność względną jest możliwa przy wykorzystaniu wzorów dostarczonych w nocie katalogowej [4]. Dla temperatury podano wzór:

$$T [^{\circ}\text{C}] = \frac{S_T}{2^{20}} \cdot 200 - 50 \quad (2.1)$$

gdzie:

- T — temperatura w stopniach Celsjusza,
- S_T — surowa wartość temperatury odczytana z czujnika.

Dla wilgotności względnej podano wzór:

$$RH [\%] = \frac{S_{RH}}{2^{20}} \cdot 100\% \quad (2.2)$$

gdzie:

- RH — wilgotność względna,
- S_{RH} — surowa wartość wilgotności względnej odczytana z czujnika.

Funkcje odpowiedzialne za konwersję przedstawiono na listingu 2.3. Wzory (2.1) i (2.2) zostały przekształcone w sposób niewymagający arytmetyki zmiennoprzecinkowej — dzielenie przez 2^{20} zastąpiono przesunięciem bitowym w prawo o 20 pozycji i wykonuje się ono dopiero po przemnożeniu surowej wartości. Wprowadzono ponadto stałą czasu kompilacji `PRECISION`, pozwalającą na skalowanie wyników i ograniczenie strat wynikających z arytmetyki całkowitoliczbowej. Przekazanie wartości `PRECISION = 1000` skutkuje zwróceniem wyników w m°C dla funkcji `convert_temperature` oraz w tysięcznych częściach procenta dla funkcji `convert_humidity`.

Listing 2.3: Konwersja surowych danych z modułu AHT20

```

1 pub fn convert_temperature<const PRECISION: i64>(raw: i64) -> i16 {
2     (((raw * 200 * PRECISION) >> 20) - 50 * PRECISION) as i16
3 }
4

```

```

5 pub fn convert_humidity<const PRECISION: i64>(raw: i64) -> i16 {
6     ((raw * 100 * PRECISION) >> 20) as i16
7 }

```

Integralność odebranych danych może zostać zweryfikowana za pomocą 8-bitowej sumy kontrolnej CRC, znajdującej się w siódmym bajcie bufora odczytu. Weryfikacja realizowana jest przez funkcję `check_crc`, przedstawioną na listingu 2.4. [TODO: Dopytaj czy opis potrzebny]

Listing 2.4: Sprawdzanie sumy kontrolnej danych modułu AHT20

```

1 fn check_crc(bytes: &[u8; 7]) -> bool {
2     let mut crc: u8 = 0xFF;
3     for &byte in &bytes[..6] {
4         crc ^= byte;
5         for _ in 0..8 {
6             if crc & 0x80 != 0 {
7                 crc = (crc << 1) ^ 0x31;
8             } else {
9                 crc <<= 1;
10            }
11        }
12    }
13    crc == bytes[6]
14 }

```

Implementacja struktury `Aht20` została uzupełniona o funkcję `read` — przedstawioną na listingu 2.5 — wykonującą surowy pomiar, weryfikację integralności danych oraz ich konwersję. Surowe wartości temperatury i wilgotności są 20-bitowymi liczbami całkowitymi bez znaku, które przed przetwarzaniem zostają rozszerzone do typu `i64`. Wybór tego typu podyktowany jest koniecznością uniknięcia przepełnienia podczas pośrednich operacji arytmetycznych. Należy zaznaczyć, że typ `i64` nie jest natywnie wspierany przez architekturę AVR i musi być realizowany programowo, co negatywnie wpływa na wydajność kodu. Wartości zwracane przez funkcję podane są z dokładnością do setnych części procenta i stopnia Celsjusza, opakowane wraz z flagą integralności danych w strukturę `Aht20Data`. [TODO: może nie dodawaj całości funkcji tylko ciekawe fragmenty]

Listing 2.5: Odczyt danych modułu AHT20

```

1 pub fn read<D: DelayNs>(
2     &self, i2c: &mut I2c,
3     delay: &mut D
4 ) -> Result<Aht20Data, Aht20Error> {
5     let raw = self.read_raw(i2c, delay)?;

```

```
6
7     let crc_passed = Self::check_crc(&raw.bytes);
8
9     // Humidity: bits [39:20] of the data payload (bytes 1-3)
10    let raw_humidity = 0i64
11        | ((raw.bytes[1] as i64) << 12)
12        | ((raw.bytes[2] as i64) << 4)
13        | ((raw.bytes[3] as i64) >> 4);
14
15    // Temperature: bits [19:0] of the data payload (bytes 3-5)
16    let raw_temperature = 0i64
17        | ((raw.bytes[3] as i64 & 0x0F) << 16)
18        | ((raw.bytes[4] as i64) << 8)
19        | (raw.bytes[5] as i64);
20
21    Ok(Aht20Data {
22        temperature:
23            Self::convert_temperature::<100>(raw_temperature),
24        humidity:
25            Self::convert_humidity::<100>(raw_humidity),
26        crc_passed,
27    })
28 }
```

2.3.2 Moduł BMP280

2.3.3 Moduł VEML7700

2.3.4 Moduł z licznikiem Geigera-Müllera

Obsługa licznika odbywa się za pośrednictwem struktury `GeigerCounter`. Sprzętowy licznik T1 mikrokontrolera skonfigurowany jest do zliczania impulsów zewnętrznych generowanych przez lampę [AI mówi rurę] Geigera-Müllera. Co sekundę zliczona wartość jest odczytywana i zapisywana do 60-elementowego bufora cyklicznego.

Funkcja `cpm` zwraca liczbę zliczeń na minutę poprzez ekstrapolację zsumowanych wartości bufora na pełną minutę, nawet w przypadku niepełnego zapełnienia bufora. Takie podejście ma dwie konsekwencje: przy czasie pracy krótszym niż minuta wynik może być obciążony większym błędem, natomiast przy pełnym buforze nagłe skoki wartości są wygładzane przez uśrednianie.

Kod odpowiedzialny za odczyt i reset licznika T1 przedstawiono na listingu 2.6. W celu zabezpieczenia przed przerwaniem wykonywania kodu zastosowano funkcję `interrupt::free` z biblioteki `avr_device`.

Listing 2.6: Odczyt zliczeń licznika T1 mikrokontrolera

```
1 fn timer1_read_and_reset(tc1: &TC1) -> u16{
2     avr_device::interrupt::free(|_| unsafe {
3         let val = tc1.tcnt1().read().bits();
4         tc1.tcnt1().write(|w| w.bits(0));
5         val
6     })
7 }
```

2.4 Komunikacja radiowa

2.4.1 Serializacja danych

2.4.2 Zabezpieczenie danych

[Nie zaimplementowane]

2.4.3 Transmisja danych

Rozdział 3

Testy

3.1 Zasięg komunikacji radiowej

3.2 Pobór prądu

3.3 Czas pracy

Podsumowanie

Tu treść podsumowania (którą — oczywiście — też piszemy na końcu).

Spis listingów

2.1	Inicjalizacja modułu AHT20	16
2.2	Odczyt surowych danych z modułu AHT20	16
2.3	Konwersja surowych danych z modułu AHT20	17
2.4	Sprawdzanie sumy kontrolnej danych modułu AHT20	18
2.5	Odczyt danych modułu AHT20	18
2.6	Odczyt zliczeń licznika T1 mikrokontrolera	20

Spis tabel

1.1	Napięcie wyjściowe U_{Wy} regulatora bocznikowego przy użyciu diody Zenera i diody LED, dla prądu emitera I_E tranzystora zwierającego. Do testu użyto zasilacza stałoprądowego.	9
1.2	Opis funkcji wyprowadzeń mikrokontrolera. Typy portów: I/O - cyfrowy port wejścia/wyjścia, Tn - podłączenie do wewnętrznego licznika n mikrokontrolera, ADC - wejście konwertera analogowo-cyfrowego. . . .	11

Spis rysunków

1.1	Schemat elektryczny części sprzętowej projektu.	8
1.2	Sekcja ładowania superkondensatorów z użyciem diod zenera (D1, D2) . .	9
1.3	Sekcja ładowania superkondensatorów z użyciem diod LED (D1, D2) . .	9
1.4	Układ regulacji napięcia oparty o przetwornicę step-up i regulator liniowy.	10
1.5	Mikrokontroler ATmega328P widoczny na schemacie projektu.	11
1.6	Tranzystor załączający linię zasilania urządzeń magistrali I^2C	12
1.7	Konwertery poziomów logicznych dla linii SDA i SCL magistrali I^2C . .	12

Bibliografia

- [1] Avr uart baud rate calculator.
- [2] Arduino. Arduino uno rev3.
- [3] Arduino. Nano | arduino documentation.
- [4] Asair - Guangzhou Aosong Electronics Co., Ltd. *AHT20 Product manuals*, 2019.
- [5] Atmel Corporation. *ATmega328P*, 2015.
- [6] Holtek Semiconductor Inc. *HT73XX Low Power Consumption LDO*.
- [7] Nanjing Micro One Electronics Inc. *DC/DC Step up Converter ME2108 Series*.
- [8] Emmanuel Pameté, Lukas Köps, Fabian Alexander Kreth, Sebastian Pohlmann, Alberto Varzi, Thierry Brousse, Andrea Balducci, and Volker Presser. The many deaths of supercapacitors: Degradation, aging, and performance fading. *Advanced Energy Materials*, 13(29):2301008, 2023.
- [9] NXP Semiconductors. *UM10204 I2C-bus specification and user manual*. NXP Semiconductors, 10 2021.